

3. **Interoperability.** At least one demonstrated case of two independently developed libraries (e.g., an HTTP library and a database driver) composing through the protocol without glue code.
4. **Frame allocator robustness.** The thread-local frame allocator mechanism produces correct behavior on all three major platforms without platform-specific workarounds.
5. **Developer comprehension.** Developer surveys or conference feedback show that the two-call launch syntax and the TLS propagation mechanism are understood by a majority of C++ developers who use coroutines for I/O.

9.6 Retrospective Commitment

We commit to a retrospective at two standard releases or six years after standardization, whichever comes first. The retrospective must answer:

1. Did major implementations ship? If not, what blocked them?
2. Did independent libraries adopt the protocol? If not, what prevented interoperability?
3. Did the thread-local frame allocator mechanism prove robust across platforms?
4. Did the two-call launch syntax prove acceptable to users, or did workarounds proliferate?
5. Did the single template parameter on `task<T>` remain sufficient as the ecosystem grew?
6. Did the vtable overhead of `executor_ref` remain negligible relative to I/O latency?
7. What design limitations emerged in practice that were not anticipated?

9.7 Prediction Registry

#	Prediction	Criterion	Revisit
1	The <i>ioAwaitable</i> protocol is sufficient for all byte-oriented I/O domains	Implementation attempts in file I/O, database I/O, and IPC succeed without protocol extensions	+3 years
2	The thread-local frame allocator achieves within 10% of benchmarked performance on all platforms	Benchmark comparison on libstdc++, libc++, MSVC STL	+2 releases
3	The two-call launch syntax does not become a barrier to adoption	Developer survey shows >60% find it acceptable or transparent	+3 years
4	The vtable overhead of <code>executor_ref</code> remains below 5% of total I/O operation cost	Microbenchmark on representative workloads across platforms	+2 releases

#	Prediction	Criterion	Revisit
5	The single template parameter on <code>task<T></code> remains sufficient for the ecosystem	No widely adopted library requires a second template parameter for environment or executor type	+5 years
6	At least three independent libraries adopt the protocol within five years	Count of libraries on vcpkg/Conan that declare <i>ioAwaitable</i> conformance	+5 years

10. Suggested Straw Polls

SG4 polled at Kona (November 2023) on [P2762R2](#) “Sender/Receiver Interface For Networking”^[9]:

“Networking should support only a sender/receiver model for asynchronous operations; the Networking TS’s executor model should be removed”

<i>SF</i>	<i>F</i>	<i>N</i>	<i>A</i>	<i>SA</i>
5	5	1	0	1

Consensus.

The approach described in this paper - a coroutine-native I/O model using C++20 language features - was not among the alternatives considered.

Poll 1. A coroutine-native I/O model is a distinct approach from both the Networking TS executor model and the sender/receiver model.

Poll 2. New research into coroutine-native I/O, not available at the time of the Kona poll, warrants consideration.

11. Thoughts on Wording

Non-normative note. The wording below is not primarily intended for standardization. Its purpose is to demonstrate how a use-case-first design produces a lean specification footprint, and to show how compact an execution model becomes when designed specifically for I/O workloads.